RAG-Agent

AI Agent with Tool Use
Architecture & Usage Guide

LangGraph | Anthropic Claude | FastAPI | ChromaDB

Persistent Checkpointing | Human-in-the-Loop | 6 Real Tools

---

github.com/lightskinhorti/AI-Agent-With-Tool-Use

Table of Contents

1. System Overview

RAG-Agent is a production-ready autonomous research agent that receives a complex question or task and autonomously decides which tools to use, in what order, and when the task is sufficiently complete.

Unlike a standard RAG pipeline (retrieve -> generate), this agent reasons in a multi-step loop: it plans, executes tools, reviews results, and can replan with a different strategy if needed. It also pauses for human approval before executing sensitive operations.

What makes this different from a chatbot?

- Multi-step reasoning with explicit reflection and replanning
- 6 real tools with Pydantic validation, retry logic, and structured logging
- Persistent state (SqliteSaver) - survives process restarts
- Human-in-the-Loop via LangGraph interrupt_before mechanism
- Full observability: token usage, tool latencies, success rates
- Automated evaluation framework with 20 test cases

2. Architecture

The system consists of four layers:

Layer 1: LangGraph StateGraph (Core)

A directed graph with 5 nodes: Planner, Executor, Reviewer, HITL Gate, and Finalizer. Each node is an async function that receives the full agent state and returns a partial update. Conditional edges route execution based on the agent's status.

Layer 2: Tool System

6 tools, each inheriting from BaseTool. Every tool has: a Pydantic input schema for validation, retry logic with exponential backoff (tenacity, 3 attempts), structured logging with structlog, and latency measurement. Tools register in a central TOOL_REGISTRY.

Layer 3: FastAPI REST API

Async API with endpoints for running agents, polling status, approving/rejecting HITL pauses, and retrieving execution traces. Agent tasks run as asyncio.create_task in the background.

Layer 4: Streamlit UI

Chat interface with real-time status polling, HITL approval panel, execution trace viewer, and observability metrics. Communicates only with the API (never with the database directly).

3. Agent Flow

The agent follows this flow for every task:

Step 1: Planning

The Planner node calls Claude with the task and descriptions of all available tools. It generates a JSON execution plan: a list of steps with tool name, arguments, and reasoning.

```
[
  {"action": "rag_search", "args": {"query": "AI regulation"}, "reasoning": "Search knowledge base"},
  {"action": "web_search", "args": {"query": "EU AI Act 2025"}, "reasoning": "Get latest info"},
  {"action": "report_writer", "args": {"title": "AI Regulation"}, "reasoning": "Compile report"}
]
```

Step 2: Execution

The Executor iterates through plan steps. For each step, it looks up the tool in the registry, validates arguments, and executes. If the tool requires HITL (like code_executor), execution pauses.

Step 3: Review

The Reviewer evaluates accumulated results. It decides: 'complete' (sufficient results), 'replan' (try a different approach), or 'continue' (more steps remain). It also generates an explicit reflection for auditability.

Step 4: Finalization

The Finalizer synthesizes all tool results and reflections into a comprehensive final answer using Claude. It also computes observability metrics: total tokens, tool latency, success rate.

4. Tool System

Every tool follows the BaseTool pattern:

```
class BaseTool(ABC):
    name: str
    description: str
    requires_human_approval: bool = False

    def get_schema() -> type[BaseModel]    # Pydantic input validation
    async def _execute(**kwargs) -> str     # Core logic (override this)
    async def run(**kwargs) -> ToolResult   # Validate + execute + log
    def to_langchain_tool()                 # Convert for LLM binding
```

Tool Details

Tool	Input	Description	HITL
rag_search	query, top_k	Hybrid ChromaDB + BM25 search	No
web_search	query, max_results	DuckDuckGo web search	No
document_fetch	url	HTML/PDF download + parsing	No
code_executor	code, description	RestrictedPython sandbox	YES
report_writer	title, sections, content	Markdown report with TOC	No
memory_store	action, key, value	SQLite key-value store	No

Hybrid Search (rag_search)

The rag_search tool combines two retrieval strategies using Reciprocal Rank Fusion (RRF):

1. Dense retrieval: ChromaDB with cosine similarity on embeddings
2. Sparse retrieval: BM25 keyword matching on tokenized documents

RRF formula: score(d) = sum(1 / (k + rank)) across both rankings, where k=60.
This produces better results than either method alone.

5. State Management

The AgentState is a TypedDict with Annotated fields. LangGraph uses the annotations to determine how to merge state updates from nodes.

Field	Type	Reducer	Purpose
messages	list[BaseMessage]	add_messages	Conversation history
tool_calls	list[ToolCallRecord]	operator.add	Accumulates tool calls
tool_results	list[ToolResultRecord]	operator.add	Accumulates results
reflections	list[str]	operator.add	Reviewer insights
plan	list[dict]	replace	Current execution plan
requires_human	bool	replace	HITL trigger flag
error_count	int	replace	Circuit breaker (max 3)
metadata	dict	replace	Token usage, latencies

6. HITL (Human-in-the-Loop)

Human-in-the-Loop is implemented using LangGraph's interrupt_before mechanism. This is a production-grade pattern used in enterprise systems for compliance and safety.

How it works

- 1. Executor detects tool requires human approval (code_executor)
- 2. Sets requires_human=True, stores pending_tool_call in state
- 3. Graph transitions toward hitl_gate node
- 4. interrupt_before=['hitl_gate'] suspends execution BEFORE entering the node
- 5. Full state is checkpointed to SQLite (process can die here)
- 6. API returns status='waiting_human' with the pending tool call details
- 7. Human reviews and calls POST /agent/approve or /agent/reject
- 8. graph.aupdate_state() injects human_feedback into checkpoint
- 9. graph.ainvoke(None) resumes from the exact saved state
- 10. hitl_gate node reads feedback: approved -> execute, rejected -> finalize

This is NOT a synchronous confirmation dialog. The process can crash between steps 5 and 7, and the agent will still resume correctly from the SQLite checkpoint.

7. API Reference

Method	Path	Body	Description
POST	/agent/run	{"task": "..."}	Start agent task (async)
GET	/agent/status/{run_id}	-	Poll execution status
GET	/agent/result/{run_id}	-	Get final answer + metrics
GET	/agent/trace/{run_id}	-	Full execution trace
POST	/agent/approve/{run_id}	{"feedback": ""}	Approve HITL pause
POST	/agent/reject/{run_id}	{"feedback": ""}	Reject HITL pause
GET	/health	-	Health check

Status Response

```
{
  "run_id": "uuid",
  "status": "executing",    // planning|executing|reviewing|waiting_human|complete|error
  "current_step": 2,
  "total_steps": 4,
  "requires_human": false,
  "pending_tool_call": null,
  "tool_calls_made": 2
}
```

8. Streamlit UI

The Streamlit frontend provides four components:

- Chat Interface: Send tasks and view agent responses
- Status Polling: Real-time progress updates (2-second interval)
- HITL Panel: Review pending code, approve or reject execution
- Sidebar: Execution trace timeline, token/latency metrics, reflections

The UI communicates exclusively with the FastAPI backend via HTTP requests. It never accesses SQLite or ChromaDB directly. This single-writer pattern prevents database locking issues.

9. Observability & Metrics

Every agent run captures these metrics:

Metric	Source	Description
total_input_tokens	LLM response	Accumulated input tokens
total_output_tokens	LLM response	Accumulated output tokens
total_tokens	Computed	Sum of input + output
total_tool_latency_ms	Tool execution	Sum of all tool durations
tool_success_rate	Computed	Successful / total tool calls
total_tool_calls	Counted	Number of tools executed
tools_used	Collected	Unique tool names used

All tool executions are logged with structlog including: tool name, execution duration, success/failure, and error details. Logs use structured JSON format in production.

10. Evaluation Framework

The evaluation framework runs 20 predefined test cases across 5 categories and measures tool selection accuracy, keyword recall, latency, and token usage.

Category	Cases	Expected Tools	What it tests
rag_retrieval	4	rag_search	Knowledge base search quality
web_search	4	web_search	Web information retrieval
multi_tool	4	Multiple	Multi-step reasoning
code_execution	4	code_executor	Computational tasks
memory_and_report	4	memory/report	Persistence + reporting

Metrics Computed

- Pass rate: Overall and per category
- Tool selection accuracy: Did the planner choose the right tools?
- Keyword recall: Does the answer contain expected content?
- Average latency: Per category
- Token cost: Average tokens per task

11. Deployment (Docker)

docker-compose.yml defines two services:

```
services:
  api:      # FastAPI on port 8000 with health check
  ui:       # Streamlit on port 8501 (depends on api)

volumes:
  agent-data:  # Shared volume for SQLite + ChromaDB persistence
```

The Dockerfile uses a multi-stage build (python:3.11-slim) to minimize image size. ChromaDB runs in embedded mode (not a separate container). Health checks ensure the UI starts only after the API is ready.

12. Configuration Reference

Variable	Default	Description
ANTHROPIC_API_KEY	(required)	Anthropic API key
AGENT_MODEL	claude-sonnet-4-20250514	Claude model identifier
AGENT_MAX_STEPS	15	Max execution steps per run
HITL_TIMEOUT_SECONDS	300	Human approval timeout
CHROMA_PERSIST_DIR	./data/chroma	ChromaDB storage path
SQLITE_DB_PATH	./data/agent.db	Checkpoint database path
MEMORY_DB_PATH	./data/memory.db	Memory tool database
LOG_LEVEL	INFO	Logging level

13. Design Decisions

SqliteSaver vs MemorySaver

MemorySaver stores state in RAM. It's lost when the process restarts. SqliteSaver persists every graph transition to disk. The agent can crash mid-execution, restart, and resume from the exact step where it stopped. 95% of tutorials use MemorySaver. We use SqliteSaver because this is production, not a demo.

interrupt_before vs confirmation dialog

LangGraph's interrupt_before suspends graph execution and persists state. The graph can be resumed hours later via API. A confirmation dialog is synchronous and blocking. Enterprise systems need async, auditable approval workflows.

Hybrid search (ChromaDB + BM25)

Dense embeddings capture semantic similarity but miss exact keywords. BM25 captures exact terms but misses meaning. Reciprocal Rank Fusion combines both rankings for consistently better retrieval quality.

RestrictedPython vs subprocess

RestrictedPython runs code in-process with restricted builtins. Faster and simpler than subprocess/container sandboxing. The allowlist is explicit: math, json, datetime, statistics, collections. For production, Docker-based sandboxing would be the natural next step.

asyncio.create_task vs Celery

Agent tasks run as asyncio tasks in the same event loop. Celery would add Redis/RabbitMQ as dependencies, which is overkill for this scope. Since state is checkpointed to SQLite, crashed tasks can be resumed from the last checkpoint.

14. Example Flows

Example 1: Multi-tool research

Task: "Research the latest AI agent frameworks and create a comparison report"

- Step 1 - Planner generates: [rag_search, web_search, web_search, report_writer]
- Step 2 - rag_search('AI agent frameworks') -> returns knowledge base results
- Step 3 - web_search('AI agent frameworks 2025 comparison') -> web results
- Step 4 - web_search('LangGraph vs CrewAI vs AutoGen') -> more web results
- Step 5 - Reviewer: 'Sufficient information. Proceed to report.'
- Step 6 - report_writer generates structured Markdown report
- Step 7 - Finalizer synthesizes comprehensive answer with references

Example 2: HITL flow (code execution)

Task: "Calculate compound interest for 10000 euros at 5% for 10 years"

- Step 1 - Planner: [code_executor]
- Step 2 - Executor detects code_executor requires HITL
- Step 3 - Graph PAUSES at hitl_gate. State saved to SQLite
- Step 4 - API returns status='waiting_human' with the code to review
- Step 5 - Human reviews code, calls POST /agent/approve
- Step 6 - Graph RESUMES. code_executor runs: result = 16288.95
- Step 7 - Finalizer: 'The compound interest result is 16,288.95 euros'